

CLASSIFICATION

Text Classification

Is this spam?

Subject: Important notice!

From: Stanford University <newsforum@stanford.edu>

Date: October 28, 2011 12:34:16 PM PDT

To: undisclosed-recipients;;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

<http://www.123contactform.com/contact-form-StanfordNew1-236335.html>

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information about the new services.

© Stanford University. All Rights Reserved.

Is this spam?

Subject: Important notice!

From: Stanford University <newsforum@stanford.edu>

Date: October 28, 2011 12:34:16 PM PDT

To: undisclosed-recipients;;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

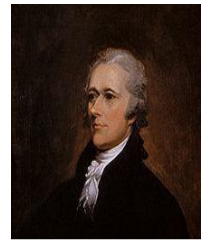
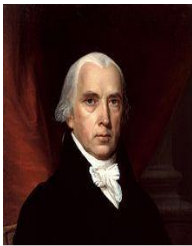
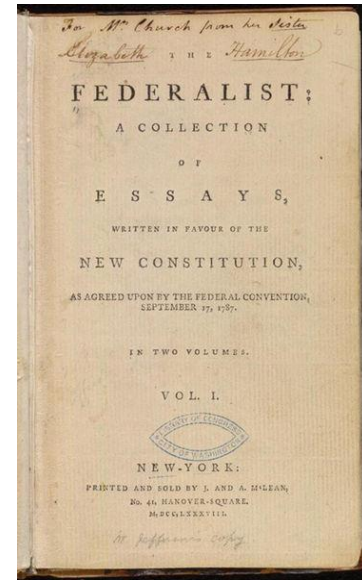
<http://www.123contactform.com/contact-form-StanfordNew1-236335.html>

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information about the new services.

© Stanford University. All Rights Reserved.

Who wrote which Federalist papers?

- 1787-8: anonymous essays try to convince New York to ratify U.S Constitution: Jay, Madison, Hamilton.
- Authorship of 12 of the letters in dispute
- 1963: solved by Mosteller and Wallace using Bayesian methods



Male or female author?

1. By 1925 present-day Vietnam was divided into three parts under French colonial rule. The southern region embracing Saigon and the Mekong delta was the colony of Cochinchina; the central area with its imperial capital at Hue was the protectorate of Annam...
2. Clara never failed to be astonished by the extraordinary felicity of her own name. She found it hard to trust herself to the mercy of fate, which had managed over the years to convert her greatest shame into one of her greatest assets...

Male or female author?

More determiners: Male

1. By 1925 present-day Vietnam was divided into three parts under French colonial rule. **The** southern region embracing Saigon and **the** Mekong delta was **the** colony of Cochinchina; **the** central area with its imperial capital at Hue was **the** protectorate of Annam...
2. Clara never failed to be astonished by the extraordinary felicity of **her** own name. **She** found it hard to trust **herself** to the mercy of fate, which had managed over the years to convert **her** greatest shame into one of **her** greatest assets...

More pronouns: Female

Positive or negative movie review?



- ...zany characters and *richly applied* satire, and some *great plot* twists



- It was *pathetic*. The worst part about it was the boxing scenes...



- ...*awesome caramel* sauce and sweet toasty almonds. I *love* this place!



- ...*awful pizza* and ridiculously overpriced...



Text Classification

- Assigning subject categories, topics, or genres
- Spam detection
- Authorship identification
- Age/gender identification
- Language Identification
- Sentiment analysis
- ...

Text Classification: definition

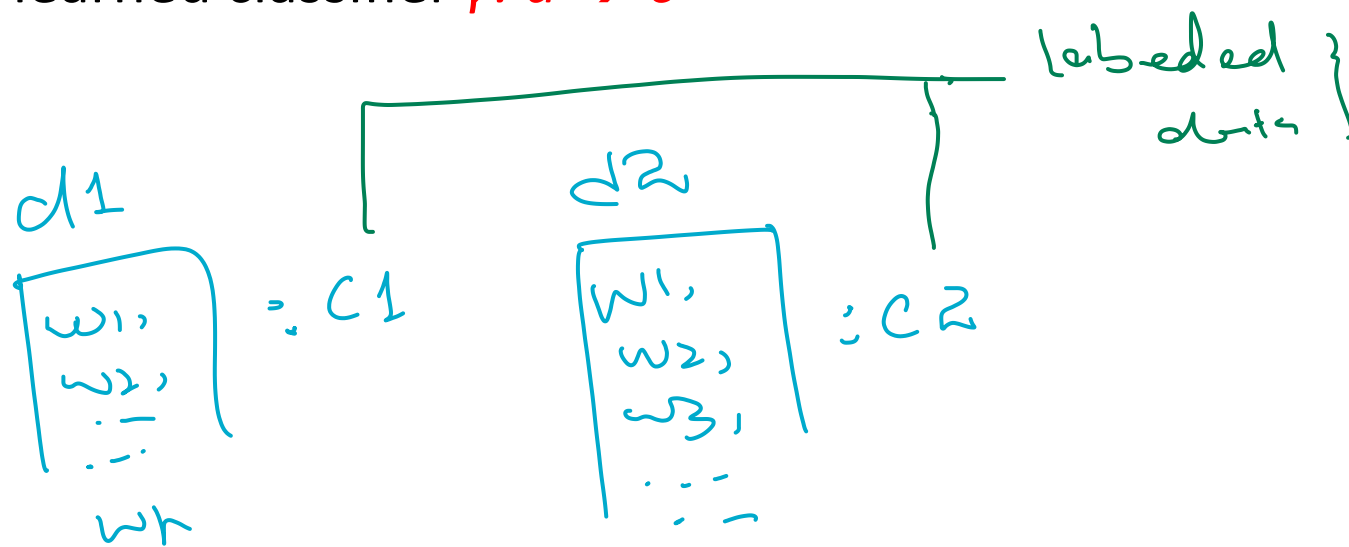
- *Input*:
 - a document d
 - a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
- *Output*: a predicted class $c \in C$

Classification Methods: Hand-coded rules

- Rules based on combinations of words or other features
 - spam: black-list-address OR (“dollars” AND “have been selected”)
- Accuracy can be high
 - If rules carefully refined by expert
- But building and maintaining these rules is expensive

Classification Methods: Supervised Machine Learning

- *Input:*
 - a document d
 - a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
 - A training set of m hand-labeled documents $(d_1, c_1), \dots, (d_m, c_m)$
- *Output:*
 - a learned classifier $\gamma: d \rightarrow c$



Classification Methods: Supervised Machine Learning

- Any kind of classifier
 - Naïve Bayes
 - Random forests
 - Logistic regression
 - Perceptrons
- **Generative classifiers** Generative classifiers model the joint distribution of features and labels, estimating both $P(X|Y)$ and $P(Y)$, and then use Bayes' theorem to compute posterior probabilities.
- like Naïve Bayes build a model for each class.
- **Discriminative classifiers** Rather than modeling the joint distribution of features and labels, discriminative classifiers focus on learning the decision boundary between classes based on the observed features.
- They aim to find the function that separates the feature space into regions corresponding to different classes. During training, the classifier learns to distinguish between spam and non-spam emails by finding the optimal decision boundary in the feature space.

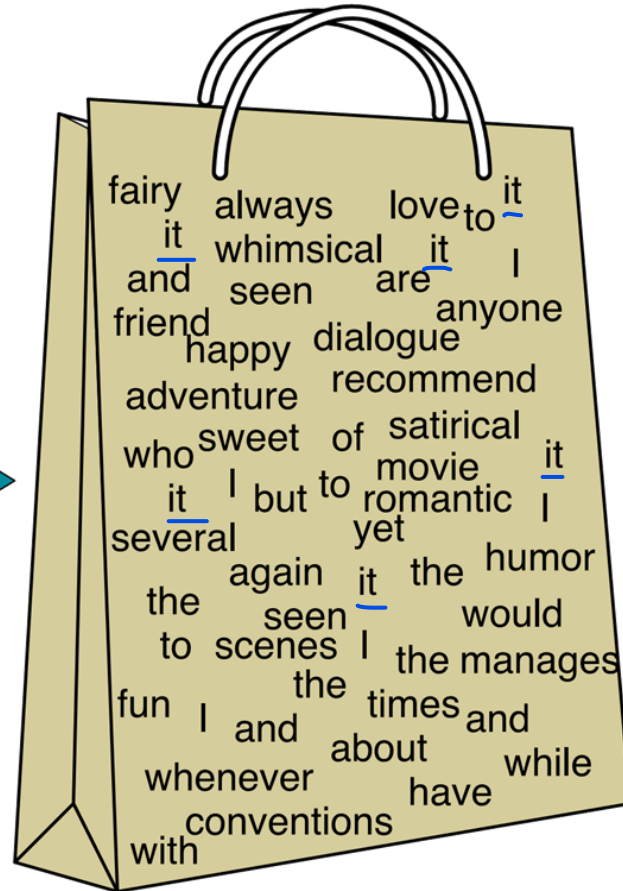
Naïve Bayes (I)

Naïve Bayes Intuition

- Classification method based on Bayes rule
- A simple (naïve) assumption about how the features interact
- Relies on a very simple representation of document
 - Bag of words

The bag of words representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



<u>it</u>	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Formalizing the Naïve Bayes Classifier

Bayes' Rule Applied to Documents and Classes

For a document *d* and a class *c*

$$P(c | d) = \frac{P(d | c)P(c)}{P(d)}$$

Naïve Bayes Classifier (I)

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(c | d)$$

Maximum a posteriori class

$C = \{c_1, c_2, c_3, \dots, c_n\}$

MAP is “maximum a posteriori” = most likely class

$$= \operatorname{argmax}_{c \in C} \frac{P(d | c)P(c)}{P(d)}$$

Bayes Rule

$P(d)$ will be same for all the class so we can ignore it

$$= \operatorname{argmax}_{c \in C} P(d | c)P(c)$$

$d = \{\dots, \text{fantastic}, \dots\}$

returns 'positive'

$C = \{\text{positive}, \text{negative}\}$

$P(\text{'fantastic'} | \text{positive})$

$P(\text{'fantastic'} | \text{negative})$

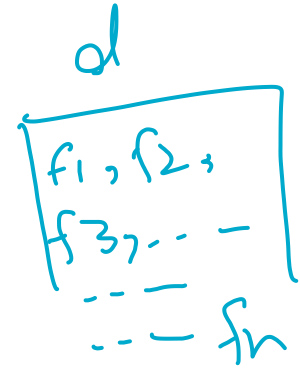
Likelihood

Prior

Dropping the denominator

Naïve Bayes Classifier (II)

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(d | c) P(c)$$



$$= \operatorname{argmax}_{c \in C} P(f_1, f_2, \dots, f_n | c) P(c)$$

Document d
represented as
features $f_1..f_n$

Naïve Bayes Classifier (IV)

$$c_{MAP} = \operatorname{argmax}_{c \in C} P(f_1, f_2, \dots, f_n | c)P(c)$$

- Hard to compute directly
- Every possible set of words and positions
- Could only be estimated if a very, very large number of training examples was available.
- Naïve Bayes makes two simplifying assumptions

Multinomial Naïve Bayes Independence Assumptions

$$P(f_1, f_2, \dots, f_n \mid c)$$

- Bag of Words assumption: Assume position doesn't matter
- Conditional Independence (aka Naïve Bayes assumption): Assume the feature probabilities $P(x_i | c_j)$ are independent given the class c .

$$P(f_1, \dots, f_n \mid c) = P(f_1 \mid c) \cdot P(f_2 \mid c) \cdot P(f_3 \mid c) \cdot \dots \cdot P(f_n \mid c)$$

Multinomial Naïve Bayes Classifier

$$\log P(c) + \dots \dots \dots \leq \sum_{f \in F} \log P(f|c)$$

$$C_{MAP} = \operatorname{argmax}_{c \in C} P(f_1, f_2, \dots, f_n | c) P(c)$$

$$P(f_1|c) * P(f_2|c) * P(f_3|c) * \dots P(f_n|c)$$

$$C_{NB} = \operatorname{argmax}_{c \in C} P(c) \prod_{f \in F} P(f | c)$$

★ to avoid underflow

argmax

$$\left[P(c_1) * \prod_{f \in F} P(f|c_1), P(c_2) * \prod_{f \in F} P(f|c_2) \right]$$

if $c_1 > c_2 \Rightarrow$ return $\boxed{c_1}$

Naïve Bayes: Learning

Learning the Multinomial Naïve Bayes Model

- First attempt: maximum likelihood estimates
 - simply use the frequencies in the data
 - fraction of documents in each class

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

$c = \{P, N\}$
 $N_{doc} = 100$
 $N_P = 70$ $N_N = 30$

- Assume a feature is just the existence of a word in the document's bag of words
- the fraction of times the word w_i appears in all documents of topic c

$\hat{P}(P) = \frac{70}{100} = 0.70$

$\hat{P}(\text{fantastic} | P)$

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} \text{count}(w, c)}$$

$\frac{30}{1000}$

- V consists of the union of all the word types in all classes, not just the words in one class c .

→ all word of V w.r.t P

Parameter estimation

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} \text{count}(w, c)} \quad \begin{array}{l} \text{fraction of times word } w_i \text{ appears among} \\ \text{all words in documents of topic } c \end{array}$$

- Create mega-document for topic c by concatenating all docs in this topic
 - Use frequency of w in mega-document

$$D_1: \{w_1, w_2 \dots w_n\} \rightarrow P$$

$$D_2: \{w_1, w_2 \dots w_n\} \rightarrow N$$

$$D_3: \{w_1, w_2 \dots w_n\} \rightarrow P$$

$$D_4: \{w_1, w_2 \dots w_n\} \rightarrow P$$

$$D_1 \cdot D_3 \cdot D_4 \rightarrow P \quad \left. \begin{array}{l} \\ \\ \end{array} \right\}$$

$$D_2 \rightarrow N$$

MegaDoc for $\{P \text{ \& } N\}$

Problem with Maximum Likelihood

- What if we have seen no training documents with the word *fantastic* in the topic **positive**?

$$\hat{P}(\text{"fantastic"} \mid \text{positive}) = \frac{\text{count}(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} \text{count}(w, \text{positive})} = 0$$

- Zero probabilities cannot be conditioned away, no matter the other evidence!

$$c_{MAP} = \operatorname{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

need some method to resolve zero probabilities. Any idea???

Laplace (add-1) smoothing for Naïve Bayes

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} (\text{count}(w, c))}$$

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} (\text{count}(w, c) + 1)}$$

$$= \frac{\text{count}(w_i, c) + 1}{\left(\sum_{w \in V} \text{count}(w, c) \right) + |V|}$$

$\sum_{w \in V} 1$
 $\rightarrow |V|$
length of
vocabulary

Unknown words

- Words that occur in our test data but not in any training document in any class
- Ignore such words i.e. remove them from the test document and **not include any probability for them at all**.
- Some systems also ignore **stop words**
 - very frequent words like *the* and *a*.
 - sort the vocabulary by frequency in the training set, and define the top 10–100 entries as stop words
 - or, use a pre-defined stop word list
 - every instance of these stop words are simply removed from both training and test documents as if they had never occurred
- However, using a **stop word list doesn't improve** performance

Algorithm

$$D_2, D_4, D_6, \dots, D_N \rightarrow c_1$$
$$D_1, D_3, D_5, \dots, D_{N-1} \rightarrow c_2$$

function TRAIN NAIVE BAYES(D, C) **returns** $\log P(c)$ and $\log P(w|c)$

for each class $c \in C$ # Calculate $P(c)$ terms

$$C = \{c_1, c_2\}$$

N_{doc} = number of documents in D

N_c = number of documents from D in class c

$$\logprior[c] \leftarrow \log \frac{N_c}{N_{doc}} \quad \frac{N_c}{N} = \frac{N}{2} \cdot \frac{1}{N} = \frac{1}{2}$$

$V \leftarrow$ vocabulary of D

$bigdoc[c] \leftarrow$ **append**(d) **for** $d \in D$ **with** class c $\rightarrow$

$$MD_c = \{D_2, D_4, \dots, D_N\}$$

for each word w in V

Calculate $P(w|c)$ terms

$$MD_c = \{D_1, D_3, \dots, D_{N-1}\}$$

$count(w, c) \leftarrow$ # of occurrences of w in $bigdoc[c]$

$$\loglikelihood[w, c] \leftarrow \log \frac{count(w, c) + 1}{\sum_{w' \in V} (count(w', c) + 1)}$$

$$V = \boxed{\begin{array}{c} \dots \\ \text{unique} \\ \text{words} \end{array}}$$

return $\logprior$, $\loglikelihood$, V

function TEST NAIVE BAYES($testdoc$, $\logprior$, $\loglikelihood$, C, V) **returns** best c

for each class $c \in C$

$sum[c] \leftarrow \logprior[c]$

for each position i in $testdoc$

$word \leftarrow testdoc[i]$

if $word \in V$

$sum[c] \leftarrow sum[c] + \loglikelihood[word, c]$

return $\argmax_c sum[c]$

Example: Sentiment analysis with add-1 smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable <u>with</u> no fun



Prior: N_c / N_{doc}

$$P(-) = \frac{3}{5} \quad P(+) = \frac{2}{5}$$

Dropping “with” as an unknown word

$$P(\text{“predictable”}|-) = \frac{1+1}{14+20} \quad P(\text{“predictable”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“no”}|-) = \frac{1+1}{14+20} \quad P(\text{“no”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“fun”}|-) = \frac{0+1}{14+20} \quad P(\text{“fun”}|+) = \frac{1+1}{9+20}$$

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$
$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

The test sentence belongs to class **negative**.

Vocabulary

1. just
2. plain
3. boring
4. entirely
5. predictable
6. and
7. lacks
8. energy
9. no
10. surprises
11. very
12. few
13. laughs
14. powerful
15. the
16. most
17. fun
18. film
19. of
20. summer

Dan Jurafsky

$$\hat{P}(c) = \frac{N_c}{N}$$

$$\hat{P}(w | c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + |V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

$$\hat{P}(c) = \frac{N_c}{N}$$

$$\hat{P}(w | c) = \frac{\text{count}(w, c) + 1}{\text{count}(c) + |V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors:

$$P(c) = \frac{3}{4}$$

$$P(j) = \frac{1}{4}$$

Choosing a class:

$$P(c | d_5) \propto \frac{3}{4} * \left(\frac{3}{7}\right)^3 * \frac{1}{14} * \frac{1}{14} \approx 0.0003$$

Conditional Probabilities:

$$P(\text{Chinese} | c) = (5+1) / (8+6) = 6/14 = 3/7$$

$$P(\text{Tokyo} | c) = (0+1) / (8+6) = 1/14$$

$$P(\text{Japan} | c) = (0+1) / (8+6) = 1/14$$

$$P(\text{Chinese} | j) = (1+1) / (3+6) = 2/9$$

$$P(\text{Tokyo} | j) = (1+1) / (3+6) = 2/9$$

$$45 \quad P(\text{Japan} | j) = (1+1) / (3+6) = 2/9$$

$$P(j | d_5) \propto \frac{1}{4} * \left(\frac{2}{9}\right)^3 * \frac{2}{9} * \frac{2}{9} \approx 0.0001$$

Sentiment Analysis: Optimization

- Whether a word occurs or not seems to matter more than its frequency
- Improves performance by clipping word counts in each document at 1
 - called binary multinomial naive Bayes or **binary NB**
 - for each document remove all duplicate words before concatenating them into the single big document
 - the word *great* has a count of 2 even for Binary NB, because it appears in multiple documents.

Sentiment Analysis: Optimization

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	–	+	–
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Sentiment Analysis: Optimization

- when a negation is present, the sentiment of the subsequent words may be reversed or altered.
- **Negation**
 - *I really like this movie* (positive)
 - *I didn't like this movie* (negative)
- Prepend the prefix *NOT* to every word after a token of logical negation (*n't, not, no, never*) until the next punctuation mark
 - *didn't like this movie , but I*
 - *didn't NOT_like NOT_this NOT_movie , but I*
- 'words' like *NOT_like, NOT_recommend* will occur more often in negative documents, while words like *NOT_bored, NOT_dismiss* will acquire positive associations

Sentiment Analysis: Insufficient data

- Insufficient labeled training data to train accurate naive Bayes classifiers
- **Sentiment lexicons**
 - Extract positive and negative word features from **sentiment lexicons**
 - lists of words that are pre-annotated with positive or negative sentiment
- MPQA subjectivity lexicon
 - 6885 words, 2718 positive and 4912 negative
 - + : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*
 - : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*
- If we do not have a lot of training data, add features:
 - **‘this word occurs in the positive lexicon’**
 - **‘this word occurs in the negative lexicon’**
 - And treat all instances of words in the lexicon as counts for that one feature, instead of counting each word separately
 - If we have lots of training data, and if the test data matches the training data, using just two features won’t work as well as using all the words

SPAM vs HAM: Optimization

Rather than using all the words as individual features:

1.Predefined Sets of Likely Features:

- Instead of using all words as individual features, predefined sets of words or phrases are identified as features.
- These sets are likely to appear frequently in either spam or ham emails. This approach reduces the dimensionality of the feature space and focuses on relevant linguistic patterns.

2.Including Non-Purely Linguistic Features:

- Features are not limited to linguistic elements alone.
- They may include non-linguistic aspects of emails, such as their structure, formatting, or metadata.
- For instance, features like HTML text-to-image ratio or the email's routing path can be considered.

SPAM vs HAM: Optimization

3. Examples from SpamAssassin:

Spam detection systems like SpamAssassin utilize predefined features such as:

- Specific phrases like "one hundred percent guaranteed" or "millions of dollars" (identified using regular expressions).
- Non-linguistic features like HTML text-to-image ratio, which can indicate spammy content.
- Consideration of the email's routing path, which provides insights into its origin and authenticity.

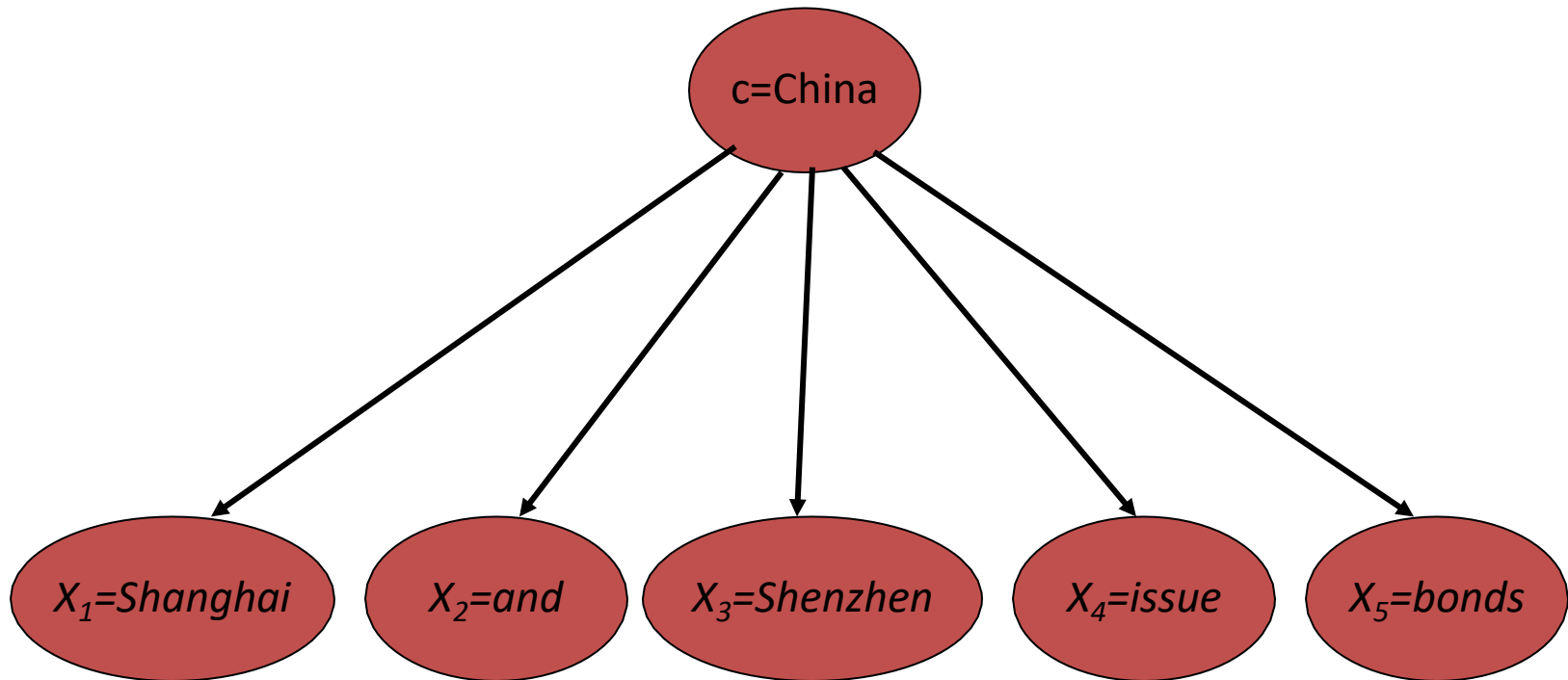
4. Other Features:

Additional features may include characteristics like:

- Email subject lines written in all capital letters, often indicative of spam.
- Presence of urgent phrases like "urgent reply" in the subject line.
- Mention of online pharmaceuticals in the email subject line or body.
- Structural irregularities in HTML, such as unbalanced "head" tags.
- Claims offering removal from mailing lists, a common tactic in spam emails.

Naïve Bayes: Relationship to Language Modeling

Generative Model for Multinomial Naïve Bayes



Naïve Bayes and Language Modeling

- Naïve bayes classifiers can use any sort of feature
 - URL, email address, dictionaries, network features
- But if, as in the previous slides
 - We use **only** word features
 - we use **all** of the words in the text (not a subset)
- Then
 - Naïve bayes has an important similarity to language modeling.

Each class = a unigram language model

- Assigning each word: $P(\text{word} \mid c)$
- Assigning each sentence: $P(s|c) = \prod_{i \in \text{positions}} P(w_i|c)$

Class	<i>pos</i>					
0.1	I	I	love	this	fun	film
0.1	love					
0.01	this	0.1	0.1	.05	0.01	0.1
0.05	fun					
0.1	film					

...

$$P(s \mid \text{pos}) = 0.00000005$$

Naïve Bayes as a Language Model

- Which class assigns the higher probability to s?

Model pos	
0.1	I
0.1	love
0.01	this
0.05	fun
0.1	film

Model neg	
0.2	I
0.001	love
0.01	this
0.005	fun
0.1	film

I	love	this	fun	film
_____	_____	_____	_____	_____
0.1	0.1	0.01	0.05	0.1
0.2	0.001	0.01	0.005	0.1

$$P(s|\text{pos}) > P(s|\text{neg})$$

Multinomial Naïve Bayes: Another Worked Example

$$\hat{P}(c) = \frac{N_c}{N}$$

$$\hat{P}(w|c) = \frac{\text{count}(w,c)+1}{\text{count}(c)+|V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors:

$$P(c) = \frac{3}{4}$$

$$P(j) = \frac{1}{4}$$

Conditional Probabilities:

$$P(\text{Chinese}|c) = (5+1) / (8+6) = 6/14 = 3/7$$

$$P(\text{Tokyo}|c) = (0+1) / (8+6) = 1/14$$

$$P(\text{Japan}|c) = (0+1) / (8+6) = 1/14$$

$$P(\text{Chinese}|j) = (1+1) / (3+6) = 2/9$$

$$P(\text{Tokyo}|j) = (1+1) / (3+6) = 2/9$$

$$P(\text{Japan}|j) = (1+1) / (3+6) = 2/9$$

Choosing a class:

$$P(c|d_5)$$

$$\propto 3/4 * (3/7)^3 * 1/14 * 1/14 \\ \approx 0.0003$$

$$P(j|d_5)$$

$$\propto 1/4 * (2/9)^3 * 2/9 * 2/9 \\ \approx 0.0001$$

Naïve Bayes in Spam Filtering

- SpamAssassin Features:
 - Online Pharmacy
 - Mentions millions of (dollar) ((dollar) NN,NNN,NNN.NN)
 - Phrase: impress ... girl
 - From: starts with many numbers
 - Subject is all capitals
 - HTML has a low ratio of text to image area
 - One hundred percent guaranteed
 - Claims you can be removed from the list
 - 'Prestigious Non-Accredited Universities'
 - http://spamassassin.apache.org/tests_3_3_x.html

Summary: Naive Bayes is Not So Naive

- Very Fast, low storage requirements
- Very good in domains with many equally important features: It excels in scenarios where multiple features are equally relevant, as it doesn't prioritize one feature over another.
- Optimal if the independence assumptions hold:
 - If assumed independence is correct, then it is the Bayes Optimal Classifier for problem
- A good dependable baseline for text classification
 - **But we will see other classifiers that give better accuracy**

Performance measures: Precision, Recall, and the F measure

Gold standards

- In spam detection (for example) for each item (email document)
 - we therefore need to know whether our system called it spam or not
 - We also need to know whether the email is actually spam or not, i.e. the human-defined labels for each document
 - We will refer to these human labels as the **gold labels**.

Gold Labels, Annotators and Agreement

- Multiple annotators

Utterance s	Ann1	Ann2	Ann Rand	Agreement (A1, Rand)	Agreement (A2, Rand)
S1	+	+	-	0	0
S2	-	-	+	0	0
S3	+	-	+	1	0
S4	-	+	-	1	0
...	...	...	...	...	...
				Sum of chance agreements	Sum of chance agreements

- **Raw agreement:** total number of agreed-upon annotations divided by the total number of possible annotations
- **Chance agreement:**
- **Cohen's Kappa and Krippendorff's alpha**
- **Agreement over a random subset with an expert annotator**

Gold Labels, Annotators and Agreement

- Chance Agreement

Example	Annotator 1	Annotator 2
1	Joy	Joy
2	Sadness	Joy
3	Anger	Anger
4	Fear	Fear
5	Joy	Sadness

- Let's consider an example of inter-annotator agreement for the task of **identifying emotions in text**. Suppose we have two annotators and five examples of text to annotate.

The frequency of each annotation is:

- Joy: 4, Sadness: 2 Anger: 2, Fear: 2
- Calculate the total number of possible annotations. Since we have two annotators and five examples, the total number of possible annotations is:

Total Number of Possible Annotations = 2 x 5 = 10

Now we can calculate the chance agreement using the formula :

Chance Agreement = Sum of (Frequency of Each Annotation / Total Number of Possible Annotations)²

$$(4/10)^2 + (2/10)^2 + (2/10)^2 + (2/10)^2 = ?$$

Cohen's Kappa

Cohen's Kappa Statistic is used to measure the level of agreement between two raters or judges who each classify items into mutually exclusive categories.

$$k = (p_o - p_e) / (1 - p_e)$$

where:

- p_o : Relative observed agreement among raters
- p_e : Hypothetical probability of chance agreement

Cohen's Kappa

Cohen's Kappa always ranges between 0 and 1, with 0 indicating no agreement between the two raters and 1 indicating perfect agreement between the two raters.

Suppose two museum curators are asked to rate 70 paintings on whether they're good enough to be hung in a new exhibit.

The following 2×2 table shows the results of the ratings:

		Rater 2	
		Yes	No
Rater 1	Yes	25	10
	No	15	20

Cohen's Kappa

Step 1: Calculate relative agreement (p_o) between raters

the proportion of total ratings that the raters both said “Yes” or both said “No” on.

We can calculate this as:

- $p_o = (\text{Both said Yes} + \text{Both said No}) / (\text{Total Ratings})$

- $p_o = (25 + 20) / (70) = \mathbf{0.6429}$

Step 2: Calculate the hypothetical probability of chance agreement (p_e) between raters

Next, calculate the probability that the raters could have agreed purely by chance.

This is calculated as the total number of times that Rater 1 said “Yes” divided by the total number of responses, multiplied by the total number of times that Rater 2 said “Yes” divided by the total number of responses, added to the total number of times that Rater 1 said “No” multiplied by the total number of times that Rater 2 said “No.”

For our example, this is calculated as:

- $P(\text{“Yes”}) = ((25+10)/70) * ((25+15)/70) = 0.285714$

- $P(\text{“No”}) = ((15+20)/70) * ((10+20)/70) = 0.214285$

- $p_e = 0.285714 + 0.214285 = \mathbf{0.5}$

Cohen's Kappa

Step 3: Calculate Cohen's Kappa

Lastly, we'll use p_o and p_e to calculate Cohen's Kappa:

- $k = (p_o - p_e) / (1 - p_e)$
- $k = (0.6429 - 0.5) / (1 - 0.5)$
- $k = 0.2857$

Cohen's Kappa turns out to be **0.2857**.

Cohen's Kappa	Interpretation
0	No agreement
0.10 - 0.20	Slight agreement
0.21 - 0.40	Fair agreement
0.41 - 0.60	Moderate agreement
0.61 - 0.80	Substantial agreement
0.81 - 0.99	Near perfect agreement
1	Perfect agreement

- Imagine you're the CEO of the *Delicious Pie Company* and you need to know what people are saying about your pies on social media
- You build a system that detects tweets concerning Delicious Pie
 - the positive class is tweets about Delicious Pie
 - the negative class is all other tweets.
- Imagine that we looked at a million tweets
 - only 100 of them are discussing their love (or hatred) for our pie
 - the other 999,900 are tweets about something completely unrelated
 - **Imagine a simple classifier that stupidly classified every tweet as “not about pie”**
 - This classifier would have 999,900 true positives and only 100 false negatives for an accuracy of $999,900/1,000,000$ or 99.99%!
- Accuracy is not a good metric when the goal is to discover something that is rare, or at least not completely balanced in frequency
- A very common situation in the world.

$$\text{Accuracy} = \frac{\text{My Correct Answers}}{\text{All Questions}} = \frac{tp + tn}{tp + tn + fp + fn}$$

(What fraction of time am I correct in my classification)

$$\text{Precision} = \frac{\text{True Positives}}{\text{My Positives}} = \frac{tp}{tp + fp}$$

(How much should you trust me when I say that something tests positive OR what fraction of my positives are true positives)

$$\text{Recall} = \text{Sensitivity} = \frac{\text{True Positives}}{\text{Real Positives}} = \frac{tp}{tp + fn}$$

(How much of the reality has been covered by my positive output? OR what fraction of the true positives is captured by my positives)

The recall for the “nothing is pie” classifier is 0

Assume:

- 10 gold and 11 copper coins are hidden in the room
- Your task to find me all gold coins and reject copper ones
- Suppose:
 - You return with 7 coins (these are your positives)
 - 5 of these are gold (these are your true positives, tp)
 - 2 are copper (these are your false positives, fp)
 - You left out 5 gold ones (false negatives, fn)
 - You correctly rejected 9 copper ones (true negatives, tn)
 - Your precision is $tp / \text{all your positives} = tp / (tp + fp) = 5/7$
 - Your recall is $tp / \text{all real positives} = tp / \text{number of actual gold coins} = tp / (tp + fn) = 5/10$
 - Your accuracy is $\text{correct answers} / \text{all attempts} = (tp + tn) / (tp + tn + fp + fn) = (5 + 9) / (5 + 9 + 2 + 5) = 14/21$
- Extremes:
 - You come back with only one coin and that is gold.
 - Your precision is very high but recall is very low
 - You return all 21 coins
 - Your recall is very high but precision very low
- We need to look at both precision and recall to find a good balanced classifier score

The 2-by-2 contingency table

	correct	not correct
selected	tp	fp
not selected	fn	tn

Precision and recall

- **Precision:** % of selected items that are correct
- Recall:** % of correct items that are selected

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 4.4 Contingency table

A combined measure of Precision and Recall

- It is useful to have a single number to describe performance
- The mean of precision and recall?
- But what kind of a mean should we use?

Means

- **Arithmetic Mean**

$$AM = \frac{a_1 + a_2 + a_3 + \cdots + a_n}{n}$$

For 2 values:

$$AM = \frac{a_1 + a_2}{2}$$

- **Geometric Mean**

$$GM = \sqrt[n]{a_1 + a_2 + a_3 + \cdots + a_n}$$

For 2 values:

$$GM = \sqrt[2]{a_1 + a_2}$$

- **Harmonic Mean**

$$HM = \frac{1}{\frac{1}{a_1} + \frac{1}{a_2} + \frac{1}{a_3} + \cdots + \frac{1}{a_n}}$$

For 2 values:

$$HM = \frac{2}{\frac{1}{a_1} + \frac{1}{a_2}} = \frac{2a_1a_2}{a_1 + a_2}$$

Properties

Ref: <http://economistatlarge.com/finance/applied-finance/differences-arithmetic-geometric-harmonic-means>

Arithmetic Mean

- The most common average
- Simplest to compute
- The AM of 2, 3 and 4 is 3
- Suitable when:
 - the data is not skewed (no extreme outliers)
 - the individual data points are not dependent on each other

Harmonic Mean

- The harmonic mean of a set of numbers is the reciprocal of the arithmetic mean of reciprocals
- Best used in situations where extreme outliers exist in the population
 - E.g. data: 35, 48, 35, 40, 50, 35, 35, 40, **150**, 35, 40, 35, 45, 45
 - AM = 47.7, HM = 41

Geometric Average

- Geometric mean should be used whenever the data points are inter-related

A combined measure

- Use harmonic mean instead of arithmetic mean as we are taking the average of ratios
- Harmonic mean *punishes* extreme values more strictly:
 - Consider a *trivial* classifier (always returns class A)
 - A very large number of data elements of class B, and a single element of class A:
 - Precision: 0.0
 - Recall: 1.0
 - Arithmetic mean = 0.5 (50% correct), despite being the *worst* possible outcome!
 - The harmonic mean is nearly 0.
 - To have a high F-score, you need *both* a high precision and high recall

F-MEASURE

- A combined measure that assesses the P/R tradeoff is F measure (weighted harmonic mean):

$$F = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2PR}{P + R}$$

- We can choose to favor precision or recall by using an interpolation weight α :

In HM we use a factor of beta, it gives more weight to the smaller of the two values being averaged.

If Beta=1 it is referred to as F1 measure

$$F = \frac{1}{\alpha \frac{1}{P} + (1-\alpha) \frac{1}{R}}$$

$$= \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

$$\beta = \frac{1-\alpha}{\alpha}$$

- Balanced F1 measure has $\beta = 1$ (that is, $\alpha = \frac{1}{2}$) as shown above.

By using the harmonic mean in the denominator of the F-measure formula, the F-measure gives equal weight to precision and recall, even when they are very different.

F-MEASURE

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

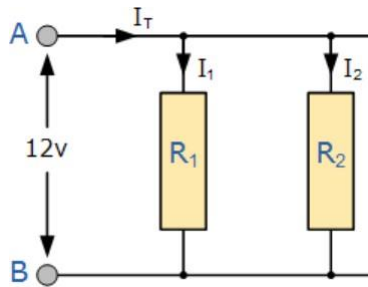
- The β parameter differentially weighs the importance of recall and precision
 - F2 measure, where recall is weighed more heavily
 - F0.5 measure, precision is weighed more heavily
- When $\beta = 1$, precision and recall are equally balanced
- This is the most frequently used metric, and is called $F_{\beta=1}$ or just $F1$:

$$F_1 = \frac{2PR}{P + R}$$

Intuition

Ref: <https://stats.stackexchange.com/questions/49226/how-to-interpret-f-measure-values>, http://www.electronics-tutorials.ws/resistor/res_4.html

- The formula for F-measure (F1, with $\beta=1$) is the same as the formula giving the equivalent resistance composed of two resistances placed in parallel in physics (forgetting about the factor 2).



$$\frac{1}{R_T} = \frac{1}{R_1} + \frac{1}{R_2}$$

$$R_T = \frac{R_1 \times R_2}{R_1 + R_2}$$

- This analogy would define F-measure as the equivalent resistance formed by *sensitivity* and *precision* placed in parallel.
- Net resistance gets reduced as soon as any one among the two loose resistance

More than two classes

More than two classes

- Lots of classification tasks in language processing have more than two classes:
 - Sentiment analysis (positive, negative, neutral),
 - Part-of-speech tagging (|POS tags|)
 - Word sense disambiguation (|word senses|)
 - Emotion detection (|emotions|)

More Than Two Classes: Sets of binary classifiers

- Any-of or multi-label classification
 - A document can belong to 0, 1, or more than one classes.
- For each class $c \in C$
 - Build a binary classifier γ_c to distinguish c from all other classes $c' \in C$ (c vs not c)
- Given test doc d ,
 - Evaluate it for membership in each class using each γ_c
 - d belongs to any class for which γ_c returns true

More Than Two Classes: Sets of binary classifiers

- One-of or multinomial classification
 - Classes are mutually exclusive: each document in exactly one class
- For each class $c \in C$
 - Build a classifier γ_c to distinguish c from all other classes $c' \in C$.
- Given test doc d ,
 - Evaluate it for membership in each class using each γ_c
 - d belongs to the one class with maximum score

Evaluation

- 3-way **one-of** email categorization decision (urgent, normal, spam)

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2) , how many documents from c_1 were (in)correctly assigned to c_2

Per class evaluation measures

Recall:

Fraction of docs in class i classified correctly:

$$\frac{C_{ii}}{\sum_j C_{ij}}$$

Precision:

Fraction of docs assigned class i that are actually about class i :

$$\frac{C_{ii}}{\sum_j C_{ji}}$$

Accuracy: (1 - error rate)

Fraction of docs classified correctly:

$$\frac{\sum_i C_{ii}}{\sum_j \sum_i C_{ij}}$$

Micro- vs. Macro-Averaging

- If we have more than one class, how do we combine multiple performance measures into one quantity?
- **Macro-averaging:** Compute performance for each class, then average.
- **Micro-averaging:** Collect decisions for all classes, compute FP and FN for each class.

Evaluation

Macro-average Precision

		gold labels			
		urgent	normal	spam	
system output	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{5+60+50}{60}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2), how many documents from c_1 were (in)correctly assigned to c_2

Class 1: Urgent			Class 2: Normal			Class 3: Spam		
	true urgent	true not		true normal	true not		true spam	true not
system urgent	8	11	system normal	60	55	system spam	200	33
system not	8	340	system not	40	212	system not	51	83
$\text{precision} = \frac{8}{8+11} = .42$			$\text{precision} = \frac{60}{60+55} = .52$			$\text{precision} = \frac{200}{200+33} = .86$		
			$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = .60$					

The **weighted-averaged** F1 score is calculated by taking the mean of all per-class F1 scores

Evaluation

Micro-average Precision

		gold labels			
		urgent	normal	spam	
system output	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2), how many documents from c_1 were (in)correctly assigned to c_2

Micro-avg precision= $\frac{TP1+TP2+TP3}{TP1+TP2+TP3+FP1+FP2+FP3}$

TP1=8, TP2= 60, TP3=200

FP1= 10+1

FP2= 50+5

FP3= 30 + 3

Micro-avg Precision= $\frac{268}{268+99}=0.73$

Evaluation

Micro-average F1 measure

		gold labels			
		urgent	normal	spam	
system output	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 6.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2), how many documents from c_1 were (in)correctly assigned to c_2

$$TP = TP1 = 8 + TP2 = 60 + TP3 = 200$$

$$TP = 268$$

$$FP = FP1 = 10 + 1 + FP2 = 50 + 5 + FP3 = 30 + 3$$

$$FP = 99$$

$$FN = 5 + 3 + 10 + 30 + 1 + 50 = 99$$

$$\text{Micro-avg Fmeasure} = 0.73$$

$$\frac{TP}{TP + \frac{1}{2} (FP + FN)}$$

Evaluation

- A micro-average is dominated by the more frequent class (in this case spam)
 - as the counts are pooled
- The macro-average better reflects the statistics of the smaller classes,
 - is more appropriate when performance on all the classes is equally important.

Evaluation

- In general, if you are working with an imbalanced dataset where all classes are equally important, using the **macro** average would be a good choice as it treats all classes equally.
- If you have an imbalanced dataset but want to assign greater contribution to classes with more examples in the dataset, then the weighted average is preferred.
 - This is because, in weighted averaging, the contribution of each class to the F1 average is weighted by its size.

Test sets and cross-validation

- We use:
 - the **training set** to train the model,
 - the **development test set** (also called a **devset**) to perhaps tune some parameters and decide what the best model is
 - Run the best model on **unseen test set** to report its performance (precision, recall, F-measure, accuracy, error rate)
- The use of a devset avoids overfitting the to test set
- But having a fixed training set, devset, and test set creates another problem:
 - in order to save lots of data for training, the test set (or devset) might not be large enough to be representative
 - It would be better if we could somehow use **all** our data both for training and test.
- We do this by **cross-validation**:
 - Multiple train test pairs
 - Randomly choose a training and test set division of our data, train our classifier, and then compute the error rate on the test set
 - Then repeat with a different randomly selected training set and test set.
 - We do this sampling process 10 times and average these 10 runs to get an average error rate.
 - This is called **10-fold cross-validation**
 - Keep one accuracy result, or take mean or std deviation of the k folds
 - Reporting values over multiple samples either through cross-validation preferred for robust evaluation

Cross-validation

- The only problem with cross-validation is:
 - Because all the data is used for testing, we need the whole corpus to be blind i.e. we can't examine any of the data to suggest possible features
 - But looking at the corpus is often important for designing the system
- It is common to create a fixed training set and test set, then do 10-fold cross-validation **inside the training set**, but compute error rate the normal way in the test set
- Comparing two classifiers
 - Cross validate with classifier A and B separately
 - Report their mean and std deviation
 - Resample the data multiple times for each classifier

Cross-validation

- Comparing two trained classifiers
 - When you only have test set
 - See the difference of the two classifiers
 - $\text{Acc}(\text{test}, A) - \text{Acc}(\text{test}, B) = 0.2$
 - A performs better than B
 - Resample the test set multiple times and then report the confidence interval
 - Resampling with replacement
 - If A performs better than B 70 times then report the results

Cross-validation with fixed test data

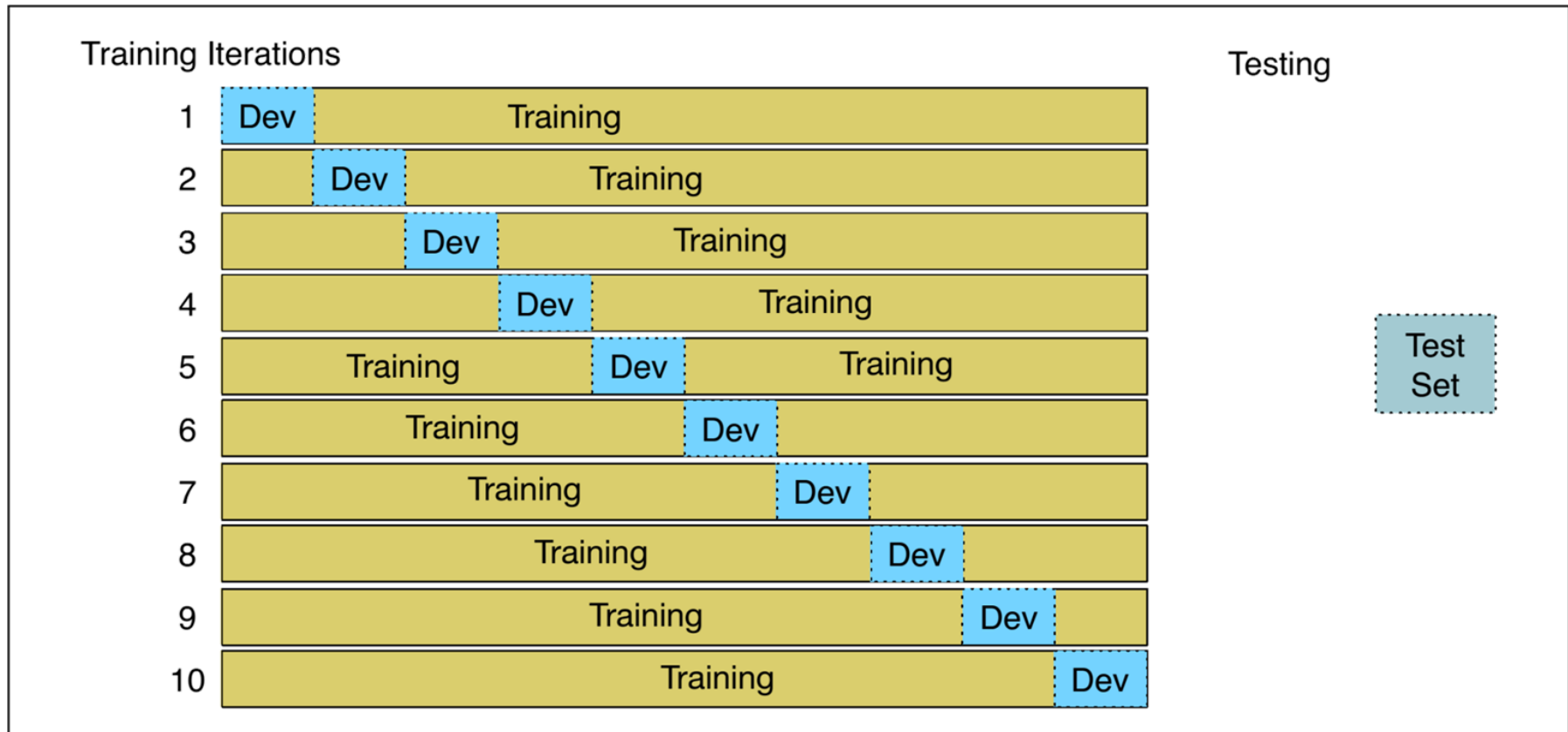


Figure 6.7 10-fold crossvalidation

Text Classification and Naïve Bayes

Text Classification: Practical
Issues